

Arquitectura x86

ARQUITECTURA DEL 8086

- En 1978 Intel presentó el procesador **8086**. Poco después, desarrolló una **variación del 8086** para ofrecer un diseño ligeramente más sencillo y compatibilidad con los dispositivos de entrada/salida de ese momento.
- Este nuevo procesador, el **8088**, fue **seleccionado por IBM para su PC en 1981**.

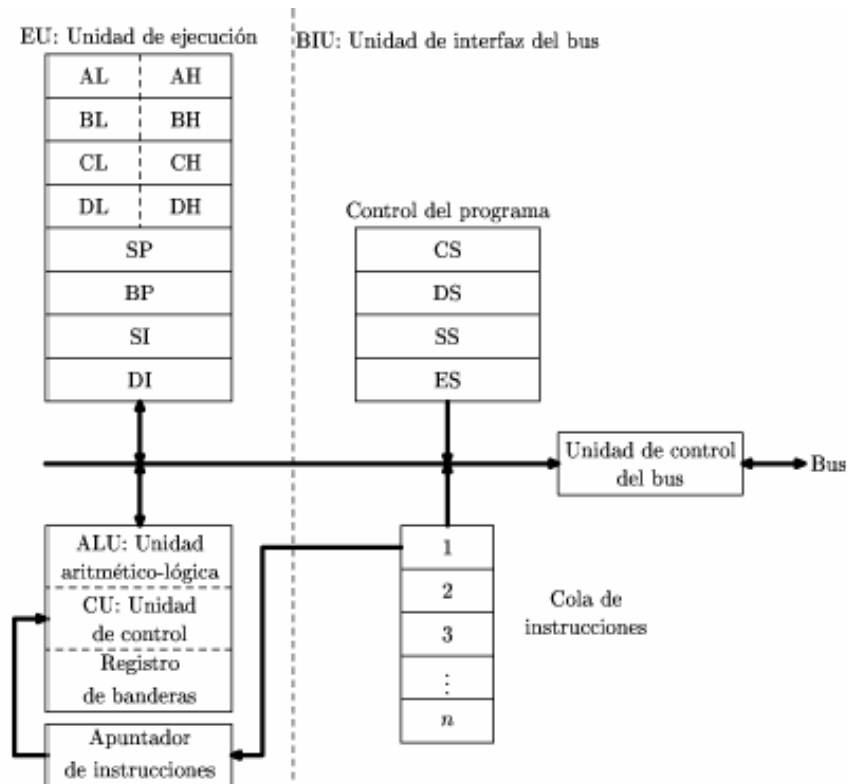
ARQUITECTURA DEL 8086

- Ambos poseen una arquitectura interna de 16 bits y pueden trabajar con operandos de 8 y 16 bits.
- Una capacidad de direccionamiento de 20 bits, es decir, existe la posibilidad de direccionar 2^{20} posiciones de memoria (bytes), lo que equivale a 1Mb de memoria principal.
- Comparten el mismo conjunto de 92 instrucciones.

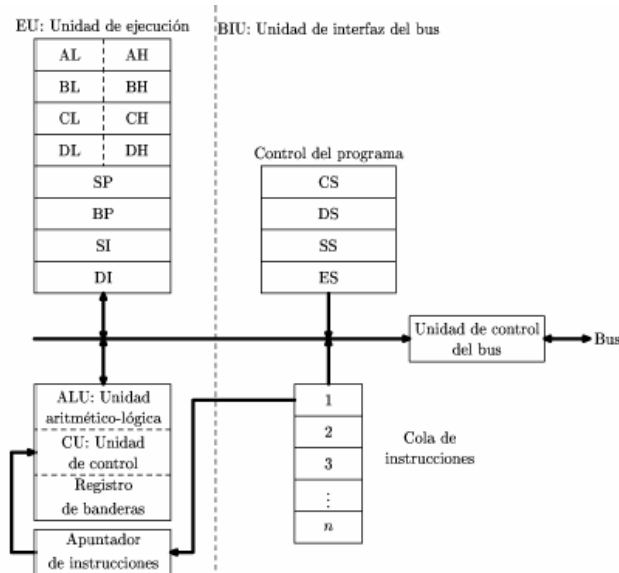
ESTRUCTURA INTERNA DEL 8086

El 8086 se divide en dos unidades lógicas.

- Una unidad de ejecución (EU) y.
- Una unidad de interfaz del bus (BIU).



ESTRUCTURA INTERNA DEL 8086



- El papel de la EU es ejecutar instrucciones, mientras que la BIU envía instrucciones y datos a la EU (unidad de ejecución).
- La EU posee una unidad aritmético-lógica, una unidad de control y 10 registros.
- Permite ejecutar las instrucciones, realizando todas las operaciones aritméticas, lógicas y de control necesarias

Registros del 8086

- Los registros del procesador tienen como misión fundamental **almacenar las posiciones de memoria** que van a sufrir repetidas manipulaciones, ya que los accesos a memoria son mucho más lentos que los accesos a los registros.
- El 8086 dispone de 14 registros de 16 bits que se emplean para controlar la ejecución de instrucciones, direccionar la memoria y proporcionar capacidad aritmética y lógica.
- Cada registro puede almacenar datos o direcciones de memoria.
- Los registros son direccionables por medio de un nombre.

Por convención los bits de un registro se numeran de derecha a izquierda

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	1	1	0	1	0	0	0	1	1	1	0

Registros del 8086

Los diferentes registros del 8086 se clasifican en:

- Registros de propósito general o de datos,
- Registros de segmento.
- Registro apuntador de instrucciones (IP).
- Registros apuntadores (SP y BP)
- Registros índice (SI y DI) y .
- Registro de banderas, FLAGS o registro de estado (FL).

AX
BX
CX
DX
Registros de propósito general o de datos

SP
BP
SI
DI
Registros punteros y Registros índice

CS
DS
SS
ES
Registros de segmento

IP
FLAGS o FL
Registro puntero de instrucciones; y Registro de banderas, FLAGS o de estado (FL)

Registros de propósito general

- Se utilizan para **cálculo y almacenamiento**.
- Los programas leen datos de memoria y los dejan en estos registros, ejecutan operaciones sobre ellos, y guardan los resultados en memoria.
- Hay cuatro registros de propósito general que, aparte de ser usados a voluntad por el programador, tienen fines específicos:

Registro AX	Este registro es el acumulador principal, implicado en gran parte de las operaciones de aritméticas y de E/S.
Registro BX	Recibe el nombre de registro base ya que es el único registro de propósito general que se usa como un índice en el direccionamiento indexado. Se suele utilizar para cálculos aritméticos.
Registro CX	El CX es conocido como registro contador ya que puede contener un valor para controlar el número de veces que se repite una cierta operación.
Registro DX	Se conoce como registro de datos. Algunas operaciones de E/S requieren su uso, y las operaciones de multiplicación y división con cifras grandes suponen que el DX y el AX trabajando juntos

Registros de Segmento.

- Los registros de segmento son registros de 16 bits.
- Para acceder a una dirección de memoria lo lógico sería colocar dicha dirección completa en un registro y acceder mediante este.
- Sin embargo, el 8086 tiene 20 líneas en el bus de direcciones (20 bits) mientras que sus registros son de 16 bits, de forma que siguiendo esta filosofía solo podríamos direccionar 64KBytes (2^{16}) del mega posible (2^{20}).

Registros de Segmento.

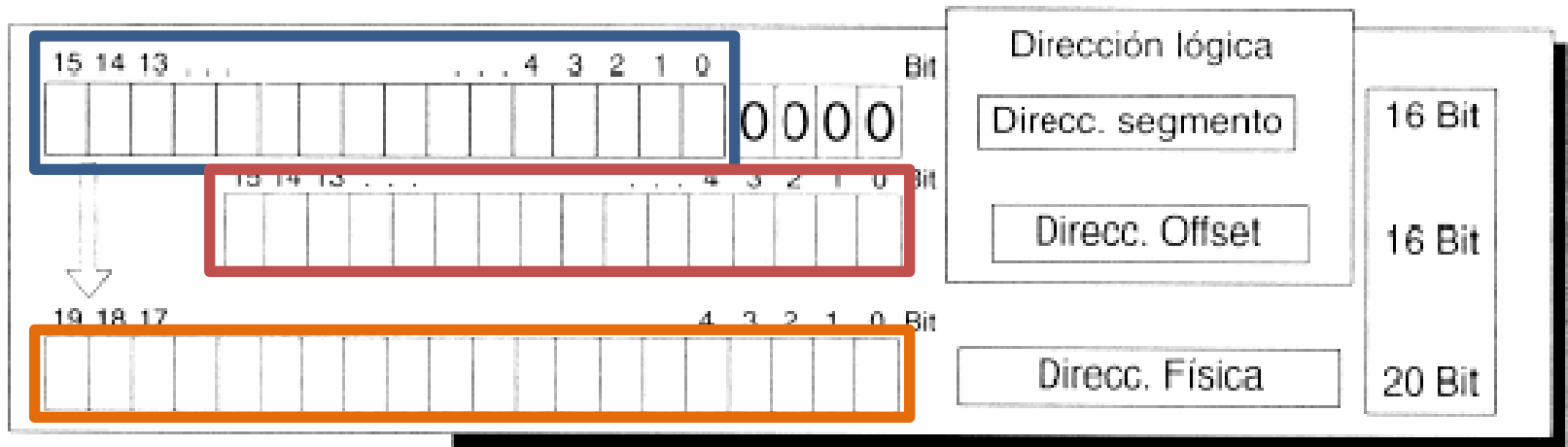
- La solución a esto consistió en usar dos registros de 16 bits, pero no dispuestos consecutivamente, ya que obtendríamos una dirección de 32 bits pasándonos del rango de memoria disponible, sino que un registro especifica un bloque de memoria, el segmento, y el otro un desplazamiento u offset dentro de este bloque.
- Al utilizar registros de 16 bits tendremos 64K posibles segmentos de 64 KBytes cada uno.

Segmentación en el 80x86

- Los segmentos no son divisiones físicas.
- Un segmento es una ventana lógica a través de la cual el programa visualiza porciones de la memoria.
- Los segmentos empiezan cada 16 bytes.
- Hay hasta 65,536 segmentos ($1,048,576/16 = 65,536$).
- El primer segmento es el segmento 0, el siguiente es el segmento 1, etc.
- El número del segmento se conoce como la dirección del segmento.
- La dirección real, también llamada dirección efectiva, en la que empieza un segmento se obtiene multiplicando la dirección del segmento por 16.

Segmentación en el 80x86

- Para direccionar cualquier posición de memoria, el procesador combina la dirección del segmento que se encuentra en un registro de segmento con un valor de desplazamiento, para calcular la dirección real de 20 bits.

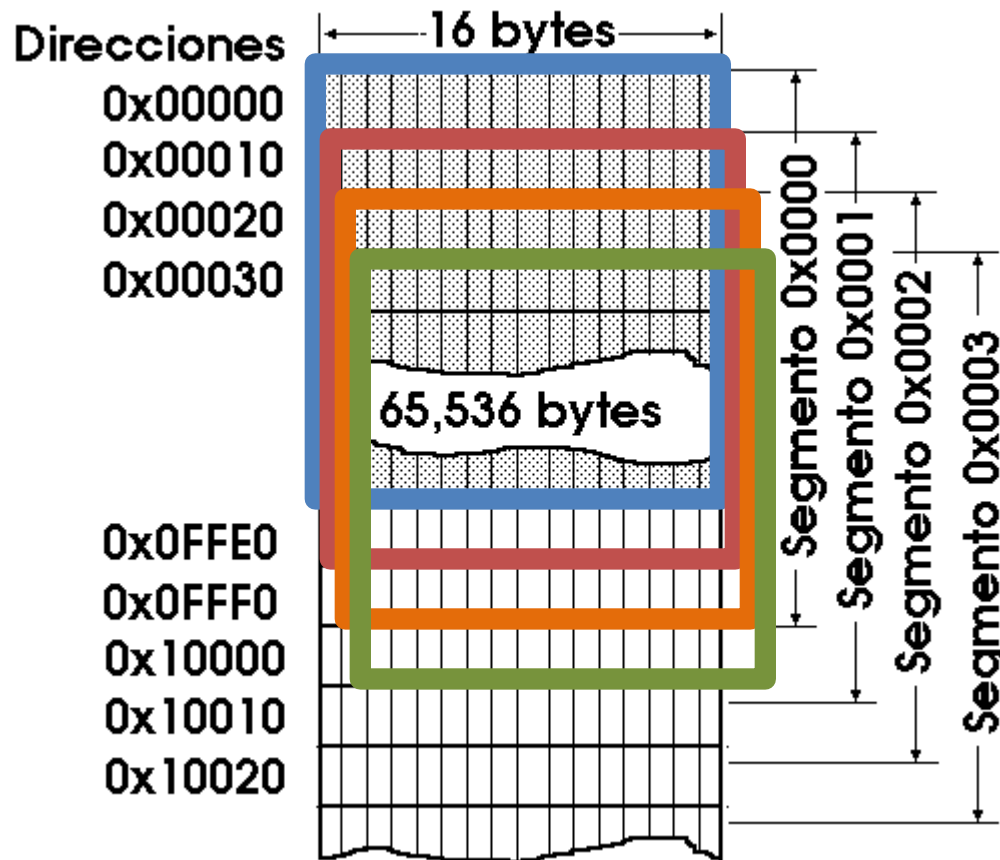


Segmentación en el 80x86

- Como vemos en la figura anterior, lo primero que se hace es añadir 4 bits a 0 al valor que se encuentra en el registro de segmento, lo cual es equivalente a multiplicar dicho valor por 16.
- Es por esto que cualquier segmento empieza siempre en una dirección de memoria (de 20 bits) múltiplo de 16, esto es, el segmento 0 comenzaría en la dirección 00000h y acabaría en la 0FFFFh.

Segmentación en el 80x86

- Por tanto, como podemos ver en la figura, los distintos segmentos se solapan, esto es, hay pares segmento-*offset* que apuntan a la misma dirección de memoria



Segmentación en el 80x86

Por ejemplo, dado el segmento 045EH y un desplazamiento de 0032H:

Dirección del segmento:	045E0H
Desplazamiento dentro del segmento:	+0032H
Dirección real:	04612H

y, dado el segmento 045FH y un desplazamiento de 0022H:

Dirección del segmento:	045F0H
Desplazamiento dentro del segmento:	+0022H
Dirección real:	04612H

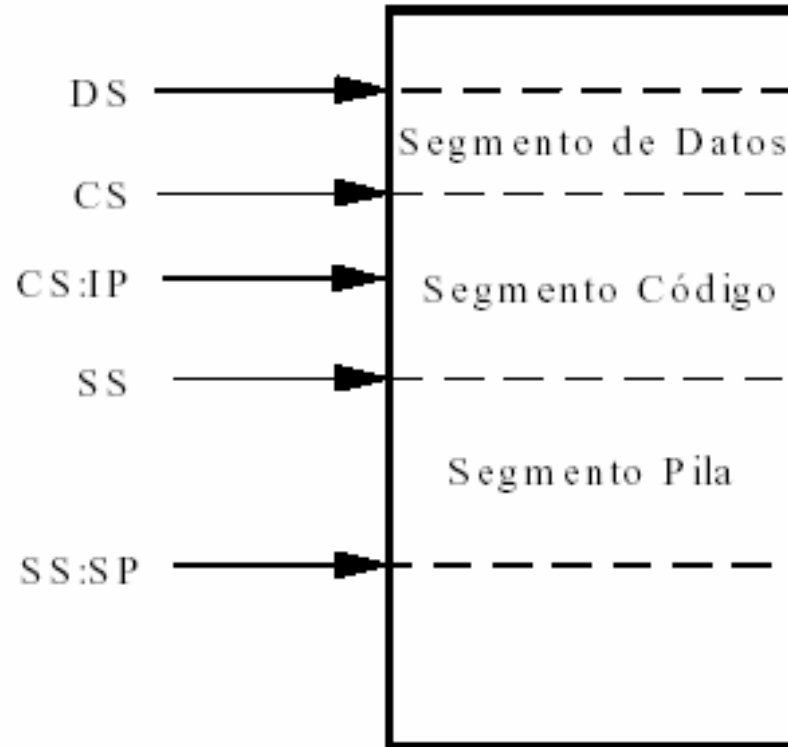
Segmentación en el 80x86

- Ahora bien, un programa puede tener uno o varios segmentos, los cuales pueden comenzar en casi cualquier lugar de la memoria, variar en tamaño y estar en cualquier orden.
- Por lo general tendrá un segmento para el código, otro para los datos y otro para la pila.
- Para apuntar a estos segmentos es para lo que sirven los registros de segmento: CS, DS, SS, ES.

Registro CS	Registro Segmento de Código. Establece el área de memoria dónde está el programa durante su ejecución.
Registro DS	Registro Segmento de Datos. Especifica la zona donde los programas leen y escriben sus datos.
Registro SS	Registro Segmento de Pila. Permite la colocación en memoria de una pila, para almacenamiento temporal de direcciones y datos.
Registro ES	Registro Segmento Extra. Para acceder a un segmento distinto de los anteriores sin necesidad de modificar los otros registros de segmento.

Segmentación en el 80x86

- Para calcular una dirección efectiva o real de un dato o una instrucción, el microprocesador toma de uno de los registros de segmento el valor del segmento y de otro registro el valor del desplazamiento.



Segmentación en el 80x86

- Para calcular una dirección efectiva o real de un dato o una instrucción, el microprocesador toma de uno de los registros de segmento el valor del segmento y de otro registro el valor del desplazamiento.
- En el cálculo de la dirección de una instrucción, el valor del segmento se encuentra en el registro de segmento de código, **CS**, y el desplazamiento en el registro apuntador de instrucciones, **IP**.
- En el cálculo de la dirección de un dato, el valor del segmento se encuentra en el registro de segmento de datos, **DS**, y el desplazamiento puede estar en los registros **BX**, **DI**, **SI**, o ser un número de 16 bits.

Segmentación en el 80x86

- En algunas instrucciones para cadenas, también se emplea el registro de segmento extra **ES**, y el registro **DI** para el cálculo de la dirección de los datos. **ES** contiene, el valor del segmento y **DI** el valor del desplazamiento.
- En el cálculo de la dirección de los datos almacenados en la pila, el valor del segmento se encuentra en el registro de segmento de pila **SS**, y el desplazamiento en el registro apuntador de pila **SP** o en el registro apuntador de base **BP**.

Registro Apuntador de Instrucciones (IP).

- Se trata de un registro de 16 bits que contiene el desplazamiento de la dirección de la siguiente instrucción que se ejecutará.
- Está asociado con el registro CS en el sentido de que IP indica el desplazamiento de la siguiente instrucción a ejecutar dentro del segmento de código determinado por CS:

Dirección del segmento de código en CS:	25A40 h
Desplazamiento dentro del segmento de código en IP:	<u>+0412 h</u>
Dirección de la siguiente instrucción a ejecutar:	25E52 h

Registros Apuntadores (SP y BP).

- Los registros apuntadores están asociados al registro de segmento de pila, SS, y permiten acceder a los datos almacenados en la pila. Representan un desplazamiento respecto a la dirección determinada por SS:

Registro SP	Proporciona un valor de desplazamiento que se refiere a la palabra actual que está siendo procesada en la pila.
Registro BP	Facilita la referencia a los parámetros de las rutinas, los cuales son datos y direcciones transmitidos vía la pila.

Registros Índice (SI y DI).

- Los registros índice se utilizan fundamentalmente en operaciones con cadenas y para direccionamiento indexado:

Registro SI	Registro índice fuente requerido en algunas operaciones con cadenas de caracteres. Este registro está asociado con el registro DS.
Registro DI	Registro índice destino requerido también en determinadas operaciones con cadenas de caracteres. Está asociado al registro DS o ES.

- Los registros de índice se pueden usar como registros de datos sin problemas para sumas, movimiento de datos, no así los de segmento, que tienen fuertes limitaciones.

Registro de Banderas, FLAGS, o registro de estado (FL)

- Es un registro de 16 bits, pero sólo se utilizan nueve de ellos.
- Sirven para indicar el estado actual de la máquina y el resultado del procesamiento.
- La mayor parte de las instrucciones de comparación y aritméticas modifican este registro.
- Algunas instrucciones pueden realizar pruebas sobre este registro para determinar la acción siguiente (como las instrucciones de salto).
- Los bits 0, 2, 4, 6, 7 y 11 son indicadores de condición que reflejan los resultados de las operaciones del programa; los bits 8 al 10 son indicadores de control que, modificados por el programador, sirven para controlar ciertos modos de procesamiento, y el resto no se utilizan.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	O	D	I	T	S	Z	-	A	-	P	-	C

Registro de banderas, FLAGS, o registro de estado (FL).

- El significado de cada uno de los bits es el siguiente:

OF	Bit de Overflow o desbordamiento. Indica desbordamiento de un bit de orden alto (más a la izquierda), después de una operación aritmética.
DF	Bit de Dirección. Designa la dirección, creciente (0) o decreciente (1), en operaciones con cadenas de caracteres.
IF	Bit de Interrupción. Indica que una interrupción externa, como la entrada desde el teclado, sea procesada o ignorada.
TF	Bit de Trap o Desvío. Procesa o ignora la interrupción interna de <i>trace</i> (procesamiento paso a paso).
SF	Bit de Signo. Indica el valor del bit más significativo del registro después de una operación aritmética o de desplazamiento.
ZF	Bit Cero. Se pone a 1 si una operación produce 0 como resultado
AF	Bit de Carry Auxiliar. Se pone a 1 si una operación aritmética produce un acarreo del bit 3 al 4. Se usa para aritmética especializada (ajuste BCD).
PF	Bit de Paridad. Se activa si el resultado de una operación tiene paridad par.
CF	Bit de Acarreo. Contiene el acarreo de una operación aritmética o de desplazamiento de bits.

Conjunto de instrucciones RISC. CISC.

- Existen dos orientaciones en cuanto a la arquitectura y funcionalidad de los procesadores actuales: RISC y CISC.

RISC (*Reduced Instruction Set Computer*)

Computadores con set de instrucciones reducido, esta arquitectura se implementa con gran éxito actualmente en microcontroladores para sistemas embebidos.

- El repertorio de instrucciones de máquina es muy reducido, menor a 60.
- Las instrucciones son simples y generalmente se ejecutan en un ciclo máquina.
- La longitud de las instrucciones tiende a ser fija y pequeña entre 12 y 32 bits.
- Tienen un número reducido de modos de direccionamiento, que son las formas como el procesador utiliza la memoria.

CISC (Complex Instruction Set Computing)

Computadores de Juego de Instrucciones Complejo. La mayoría de CPU's utilizada en microcontroladores están basados en esta arquitectura:

- Un gran número de instrucciones de longitud variable.
- Generalmente más de 80 instrucciones en su repertorio.
- Instrucciones muy sofisticadas y potentes, requiriendo muchos ciclos para su ejecución.
- Ofrecen al programador instrucciones complejas que actúan como macros.
- Modos de direccionamiento múltiple.

Set de Instrucciones (ISA) x86

El conjunto de instrucciones x86 se introdujo con el Intel 8086 y se ha ampliado varias veces a lo largo de los años. A continuación, se proporciona un resumen de las instrucciones más importantes y comunes. Se pueden dividir en las siguientes categorías:

- *Instrucciones de Transferencia de Datos*
- *Instrucciones Lógicas.*
- *Instrucciones de Desplazamiento, Rotación.*
- *Instrucciones Aritméticas*
- *Control de Bucles (instrucciones simples)*
- *Instrucciones de Prueba, Comparación y Saltos.*
- *Instrucciones de Llamado y Retorno de Subrutinas.*
- *Instrucciones de Control del microprocesador.*
- *Tratamiento de cadenas:*
- *Instrucciones de Pila.*
- *Instrucciones de Interrupción*

Instrucciones de Transferencia de Datos

Estas instrucciones mueven datos de una parte a otra del sistema, desde y hacia la memoria principal, entre registros, puertos de E/S.

- ***MOV*** transfiere
- ***XCHG*** intercambia
- ***IN*** entrada
- ***OUT*** salida
- ***XLAT*** traduce usando una tabla
- ***LEA*** carga la dirección efectiva
- ***LDS*** carga el segmento de datos
- ***LES*** carga el segmento extra
- ***LAHF*** carga los indicadores en ***AH***
- ***SAHF*** guarda ***AH*** en los indicadores
- ***PUSH FUENTE*** (*sp*) *fuentes*
- ***POP DESTINO*** *destino* (*sp*)

Instrucciones Lógicas.

Son operaciones bit a bit que trabajan sobre bytes o palabras completas (16-32 bits)

- ***NOT*** negación
- ***AND*** producto lógico
- ***OR*** suma lógica
- ***XOR*** suma lógica exclusiva

Instrucciones de Desplazamiento, Rotación.

Básicamente permiten multiplicar y dividir por potencias de 2

- ***SHL, SAL*** desplazar a la *izquierda*
(desplazamiento aritmético)
- ***SHR*** desplazar a la *derecha*
- ***SAR*** desplazamiento aritmético a la *derecha*
- ***ROL*** rotación a la *izquierda*
- ***ROR*** rotación a la *derecha*
- ***RCL*** rotación con acarreo a la *izquierda*
- ***RCR*** rotación con acarreo a la *derecha*
- ***CLC*** borrar acarreo

Instrucciones Aritméticas

a. Grupo de adición:

- ***ADD*** suma
- ***ADC*** suma con acarreo
- ***AAA*** ajuste ASCII para la suma
- ***DAA*** ajuste decimal para la suma

b. Grupo de sustracción:

- ***SUB*** resta
- ***SBB*** resta con acarreo negativo
- ***AAS*** ajuste ASCII para la resta
- ***DAS*** ajuste decimal para la resta

c. Grupo de multiplicación:

- ***MUL*** multiplicación
- ***IMUL*** multiplicación entera
- ***AAM*** ajuste ASCII para la multiplicación

d. Grupo de división:

- ***DIV*** división
- ***IDIV*** división entera
- ***AAD*** ajuste ASCII para la división

e. Conversiones:

- ***CBW*** pasar byte a palabra
- ***CWD*** pasar palabra a doble palabra
- ***NEG*** negación

Control de Bucles

Un bucle es un bloque de código que se ejecuta varias veces. Hay 4 tipos de bucles básicos:

- Bucles *sin fin*
- Bucles por *conteo*
- Bucles *hasta*
- Bucles *mientras*

Las instrucciones de control de bucles son las siguientes:

- ***INC*** incrementar
- ***DEC*** decrementar
- ***LOOP*** realizar un bucle
- ***LOOPZ, LOOPE*** realizar un bucle *si es cero*
- ***LOOPNZ, LOOPNE*** realizar un bucle *si no es cero*
- ***JCXZ*** salta si *CX* es *cero*

Instrucciones de Prueba, Comparación y Saltos

Instrucciones que evalúan condiciones y cambian el orden de ejecución según el resultado.

- ***TEST*** verifica
- ***CMP*** compara
- ***JMP*** salta
- ***JE, JZ*** salta si *es igual a cero*
- ***JNE, JNZ*** salta si *no igual a cero*
- ***JS*** salta si *signo negativo*
- ***JNS*** salta si *signo no negativo*
- ***JP, JPE*** salta si *paridad par*
- ***JNP, JOP*** salta si *paridad impar*
- ***JO*** salta si *hay capacidad excedida*
- ***JNO*** salta si *no hay capacidad excedida*
- ***JB, JNAE*** salta si *por abajo* (no encima o igual)
- ***JNB, JAE*** salta si *no está por abajo* (encima o igual)
- ***JBE, JNA*** salta si *por abajo o igual* (no encima)
- ***JNBE, JA*** salta si *no por abajo o igual* (encima)
- ***JL, JNGE*** salta si *menor que* (no mayor o igual)
- ***JNL, JGE*** salta si *no menor que* (mayor o igual)
- ***JLE, JNG*** salta si *menor que o igual* (no mayor)
- ***JNLE, JG*** salta si *no menor que o igual* (mayor)

Instrucciones de Llamado y Retorno de Subrutinas.

Para que los programas resulten eficientes y legibles tanto en lenguaje ensamblador como en lenguaje de alto nivel, resultan indispensables las subrutinas:

- ***CALL llamada*** a subrutina
- ***RET retorno*** al programa o subrutina que llamó

Instrucciones de Control del microprocesador.

Hay varias instrucciones para el control de la CPU, ya sea a ella sola, o en conjunción con otros procesadores:

- ***NOP*** no operación
- ***HLT*** parada
- ***WAIT*** espera
- ***LOCK*** bloquea
- ***ESC*** escape

Tratamiento de cadenas

Permiten el movimiento, comparación o búsqueda rápida en bloques de datos:

- ***MOVC*** transferir *carácter* de una cadena
- ***MOVW*** transferir *palabra* de una cadena
- ***CMPC*** comparar *carácter* de una cadena
- ***CMPW*** comparar *palabra* de una cadena
- ***SCAC*** buscar *carácter* de una cadena
- ***SCAW*** buscar *palabra* de una cadena
- ***LODC*** cargar *carácter* de una cadena
- ***LODW*** cargar *palabra* de una cadena
- ***STOC*** guardar *carácter* de una cadena
- ***STOW*** guardar *palabra* de una cadena
- ***REP*** repetir
- ***CLD*** poner a 0 el indicador de dirección
- ***STD*** poner a 1 el indicador de dirección

Instrucciones de Pila

Una de las funciones de la pila del sistema es la de salvaguardar (conservar) datos y la otra es la de salvaguardar las direcciones de retorno de las llamadas a subrutinas:

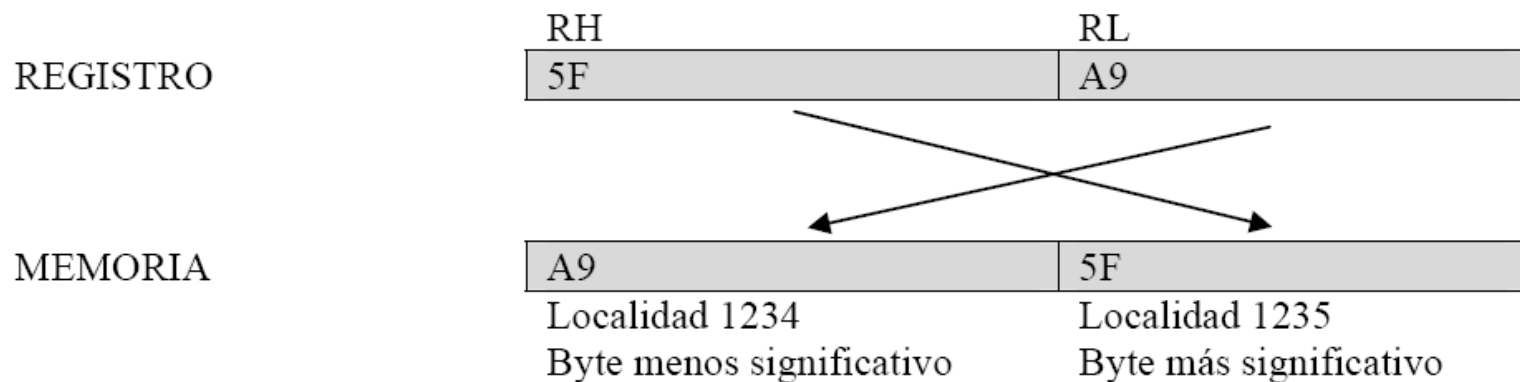
- ***PUSH*** introducir
- ***POP*** extraer
- ***PUSHF*** introducir indicadores
- ***POPF*** extraer indicadores

Instrucciones de Interrupción

- ***STI*** poner a 1 el indicador de interrupción
- ***CLI*** borrar el indicador de interrupción
- ***INT*** interrupción
- ***INTO*** interrupción por capacidad excedida (desbordamiento)
- ***IRET*** retorno de interrupción

Direccionamiento de Localidades de Memoria

- El 8086 posee un bus de datos de 16 bits y por tanto manipula cantidades de esta longitud (llamadas palabras).
- Cada palabra a la que se accede consta de dos bytes, un byte de orden alto o más significativo y un byte de orden bajo o menos significativo.
- El sistema almacena en memoria estos bytes de la siguiente manera: el byte menos significativo en la dirección baja de memoria y el byte más significativo en la dirección alta de memoria



TERMINALES EN EL 8086

